

ARQUITETURA E DESENVOLVIMENTO EM JAVA

FASE 1 | AULA 04 -

LSKOV SUBSTITUTION PRINCIPLE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS	11

EMANIP

O QUE VEM POR AÍ?

Nesta aula, vamos explorar o Princípio da Substituição de Liskov (Liskov Substitution Principle), que é o terceiro dos cinco princípios SOLID da programação orientada a objetos e design de software. Vamos entender a importância de garantir que as subclasses possam ser usadas como substitutas de suas superclasses sem alterar as propriedades desejáveis do programa.



HANDS ON

Nesta aula prática, apresentaremos o LSP (Liskov Substitution Principle) do SOLID. Aprenderemos o conceito de substituição de Liskov e os motivos pelos quais ele é fundamental para a criação de um código polimórfico e robusto.

Além disso, identificaremos violações do LSP, reconhecendo situações em que as subclasses não podem substituir suas superclasses sem alterar o comportamento esperado do programa. Por fim, vamos refatorar um código aplicando o LSP. Veremos um exemplo prático de um código refatorado de uma hierarquia de classes para que siga o Princípio da Substituição de Liskov.

SAIBA MAIS

O Princípio da Substituição de Liskov (LSP) afirma que os objetos de uma classe derivada devem poder ser substituídos por objetos da classe base sem alterar o comportamento correto do programa.

A ideia principal do LSP é garantir que uma classe derivada possa substituir sua classe base sem introduzir comportamentos inesperados ou erros. Isso significa que a classe derivada deve manter todas as garantias feitas pela classe base e não deve alterar o comportamento esperado.

Vamos considerar um exemplo de uma hierarquia de classes que representa dois tipos de contas bancárias: `ContaComum` e `ContaPremium`.

Violando o LSP

Aqui está um exemplo de uma classe `ContaComum` e uma subclasse `ContaPremium` que viola o LSP ao introduzir comportamentos inesperados:

`ContaComum`:

```
package src;

public class ContaComum {
    protected double saldo;

    public ContaComum(double saldoInicial) {
        this.saldo = saldoInicial;
    }

    public void sacar(double valor) {
        if (valor <= saldo) {
            saldo -= valor;
        } else {
            throw new IllegalArgumentException("Saldo insuficiente");
        }
    }
}
```

```
public double getSaldo() {  
    return saldo;  
}  
}
```

Código 1 — Exemplo ContaComum
Fonte: elaborado pelo autor (2024)

ContaPremium:

```
package src;  
  
public class ContaPremium extends ContaComum {  
    public ContaPremium(double saldoInicial) {  
        super(saldoInicial);  
    }  
  
    @Override  
    public void sacar(double valor) {  
        saldo -= valor; // Permite saldo negativo  
    }  
}
```

Código 2 — Exemplo ContaPremium
Fonte: elaborado pelo autor (2024)

Nesta hierarquia, a subclasse **ContaPremium** altera a lógica de saque, permitindo saldo negativo, o que não é esperado para uma **ContaComum**.

Aplicando o LSP

Para aplicar o LSP, vamos refatorar o código, garantindo que a subclasse **ContaPremium** mantenha o comportamento esperado da superclasse **ContaComum**.

ContaPremium:

```
package src;

public class ContaPremium extends ContaComum {
    public ContaPremium(double saldoInicial) {
        super(saldoInicial);
    }
    // Não sobrescreve o método sacar para manter o comportamento esperado
}
```

Código 3 — ContaPremium (2)
Fonte: elaborado pelo autor (2024)

Explicando a refatoração

Antes da Refatoração, a subclasse **ContaPremium** estava violando o LSP ao permitir saldo negativo, alterando o comportamento esperado da classe base **ContaComum**. Já depois da refatoração:

- Classe **ContaComum**: mantém a lógica de saque, garantindo que o saldo não possa ser negativo.
- Classe **ContaPremium**: não sobrescreve o método **sacar**, mantendo o comportamento esperado da superclasse **ContaComum**.

Benefícios da refatoração

- Consistência de comportamento: as subclasses mantêm o comportamento esperado da superclasse, garantindo substituição segura.
- Facilidade de manutenção: o código se torna mais previsível e fácil de manter, pois as subclasses não introduzem comportamentos inesperados.
- Robustez: garante que o polimorfismo seja aplicado corretamente, permitindo que as subclasses sejam usadas de forma intercambiável com as superclasses sem risco de erros.

Caso tenha ficado alguma dúvida sobre algum dos pontos anteriores, vamos a um breve resumo. Conforme vimos no início desta aula, a ideia principal do LSP é garantir que uma classe derivada possa substituir sua classe base sem introduzir comportamentos inesperados ou erros. Isso significa que a classe derivada deve

manter todas as garantias feitas pela classe base e não deve alterar o comportamento esperado.

No nosso exemplo de **ContaComum** e **ContaPremium**:

Inicialmente, tínhamos uma classe **ContaComum** que implementava a lógica de saque garantindo que o saldo não poderia ficar negativo. No entanto, a subclasse **ContaPremium** violava o LSP ao sobrescrever o método **sacar** para permitir saldo negativo, introduzindo um comportamento inesperado.

Para aplicar o LSP corretamente, refatoramos a subclasse **ContaPremium** para não modificar a lógica de saque da superclasse **ContaComum**. Com isso, **ContaPremium** mantém todas as garantias da classe base e pode ser usada de forma intercambiável com **ContaComum**, sem introduzir comportamentos inesperados ou erros.

Isso garante que o polimorfismo seja aplicado corretamente, permitindo que **ContaPremium** seja usada como uma substituta segura para **ContaComum**, mantendo a consistência do comportamento esperado no sistema.

Por exemplo, suponha que temos um método que processa saques de contas bancárias, esperando uma instância de **ContaComum**. Com a aplicação correta do LSP, podemos passar uma instância de **ContaPremium** para esse método sem problemas:

```
package src;

public class ProcessadorDeSaques {
    public void processarSaque(ContaComum conta, double valor)
    {
        conta.sacar(valor);
        System.out.println("Saque processado. Saldo atual: " +
        conta.getSaldo());
    }
}
```

Código 4 — Exemplo
Fonte: elaborado pelo autor (2024)

Main.java:

```
package src;

public class Main {
    public static void main(String[] args) {
        ContaComum contaComum = new ContaComum(1000);
        ContaComum contaPremium = new ContaPremium(1000);

        ProcessadorDeSaques processador = new ProcessadorDeSaques();

        // Processando saque para ContaComum
        processador.processarSaque(contaComum, 200); // Saída: Saque
        processado. Saldo atual: 800.0

        // Processando saque para ContaPremium
        processador.processarSaque(contaPremium, 200); // Saída: Saque
        processado. Saldo atual: 800.0
    }
}
```

Código 5 — Exemplo (2)
Fonte: elaborado pelo autor (2024)

Neste exemplo, o método `processarSaque` espera uma instância de `ContaComum`, mas pode receber uma instância de `ContaPremium` sem problemas. As classes `ContaComum` e `ContaPremium` mantêm o comportamento esperado de não permitir saldo negativo, garantindo a consistência do sistema e aplicando corretamente o polimorfismo.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos em profundidade o Princípio da Substituição de Liskov, o terceiro dos cinco princípios SOLID da programação orientada a objetos e design de software.



REFERÊNCIAS

CASTILHO, R. **Princípios SOLID: Princípio de Substituição de Liskov (LSP)**. 2024. Disponível em: <https://robsoncastilho.com.br/2013/03/21/principios-solid-principio-de-substituicao-de-liskov-lsp/>. Acesso em: 07 ago. 2024.

MEDIUM. **Os Princípios do SOLID — LSP Princípio da substituição de Liskov**. 2019. Disponível em: <https://medium.com/@jonesroberto/os-princ%C3%ADpios-do-solid-lsp-princ%C3%ADpio-da-substitui%C3%A7%C3%A3o-de-liskov-35bcff93cd86>. Acesso em: 07 ago. 2024.

STACKIFY. **SOLID Design Principles Explained: The Liskov Substitution Principle with Code Examples**. 2024. Disponível em: <https://stackify.com/solid-design-liskov-substitution-principle/>. Acesso em: 07 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: SOLID, Programação Orientada a Objetos, LSP.

EMENDAS



POSTECH